

```
1 using CustomProjectRPG.Interfaces;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Text.Json;
7 using System.Text.Json.Serialization;
8 using System.Threading.Tasks;
9
10
11 namespace CustomProjectRPG
12 {
13
14     public class FavorComponent: ISerializeJSON<FavorComponent>
15     {
16         private Dictionary<string, int> _favorMap;
17         private Dictionary<string, int> _hiddenFavorMap;
18
19         public FavorComponent()
20         {
21             _favorMap = new Dictionary<string, int>();
22             _hiddenFavorMap = new Dictionary<string, int>();
23         }
24
25         public int GetFavor(string npcId)
26         {
27             ValidateNpcId(npcId);
28             return _favorMap.ContainsKey(npcId) ? _favorMap[npcId] : 0;
29         }
30
31         public int GetHiddenFavor(string npcId)
32         {
33             ValidateNpcId(npcId);
34             return _hiddenFavorMap.ContainsKey(npcId) ? _hiddenFavorMap
35                 [npcId] : 0;
36
37         }
38
39         public void AdjustFavor(string npcId, int amount)
40         {
41             ValidateNpcId(npcId);
42             if (_favorMap.ContainsKey(npcId))
43             {
44                 _favorMap[npcId] += amount;
45             }
46             else
47             {
48                 _favorMap[npcId] = amount;
49             }
50         }
51     }
52 }
```

```
49
50     public void AdjustHiddenFavor(string npcId, int amount)
51     {
52         Validate_npcId(npcId);
53         if (_hiddenFavorMap.ContainsKey(npcId))
54         {
55             _hiddenFavorMap[npcId] += amount;
56         }
57         else
58         {
59             _hiddenFavorMap[npcId] = amount;
60         }
61     }
62
63     public bool CanUseFavor(string npcId, int cost)
64     {
65         Validate_npcId(npcId);
66         return _favorMap.ContainsKey(npcId) && _favorMap[npcId] >=  ⤴
            cost;
67     }
68
69     public bool UseFavor(string npcId, int cost)
70     {
71         if (CanUseFavor(npcId, cost))
72         {
73             _favorMap[npcId] -= cost;
74             return true;
75         }
76         return false;
77     }
78
79     public string Serialize()
80     {
81         var data = new FavorData
82         {
83             FavorMap = _favorMap,
84             HiddenFavorMap = _hiddenFavorMap
85         };
86
87         return JsonSerializer.Serialize(data);
88     }
89
90     public void Deserialize(string json)
91     {
92         if (string.IsNullOrEmpty(json)) return;
93
94         var data = JsonSerializer.Deserialize<FavorData>(json);
95         _favorMap = data?.FavorMap ?? new Dictionary<string, int>();
96         _hiddenFavorMap = data?.HiddenFavorMap ?? new  ⤴
```

```
Dictionary<string, int>();
97     }
98
99     private void Validate_npcId(string npcId)
100    {
101        if (string.IsNullOrEmpty(npcId))
102        {
103            throw new ArgumentException("NPC ID cannot be null or
104                                     empty.");
105        }
106
107        private class FavorData
108        {
109            [JsonPropertyName("favorMap")]
110            public Dictionary<string, int> FavorMap { get; set; }
111
112            [JsonPropertyName("hiddenFavorMap")]
113            public Dictionary<string, int> HiddenFavorMap { get; set; }
114        }
115    }
116 }
117
```